

CHƯƠNG 06: TÌM KIẾM ĐỐI KHÁNG VÀ TRÒ CHƠI

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 6

- ◇ 6.1 Trò chơi và Lý thuyết Trò chơi (Games and Game Theory)
- ◇ 6.2 Quyết định tối ưu: Thuật toán Minimax (Optimal Decisions)
- ◇ 6.3 Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)
- ◇ 6.4 Quyết định thời gian thực không hoàn hảo (Imperfect Real-Time Decisions)
- ◇ 6.5 Trò chơi ngẫu nhiên & Khuyết thiếu thông tin (Stochastic & Partially Observable Games)

6.1 Trò chơi và Lý thuyết Trò chơi

◇ Khác với tìm kiếm thông thường (chỉ có 1 tác nhân), **Môi trường đa tác nhân (Multi-agent environment)** đòi hỏi tác nhân phải đối mặt với các thực thể khác cũng đang cố gắng tối đa hóa mục tiêu của riêng họ.

◇ **Trò chơi Đối kháng (Adversarial Search):**

- Là trò chơi **Tổng bằng không (Zero-sum games)**: Tổng lợi ích của mọi người luôn cố định. Một bên được lợi bao nhiêu thì bên kia chịu thiệt bấy nhiêu.
- Đặc điểm: Tính tất định (Deterministic), Quan sát hoàn toàn (Fully observable), Đổi lượt luân phiên (Turn-taking).

◇ **Ví dụ kinh điển:** Cờ vua (Chess), Cờ vây (Go), Cờ caro (Tic-Tac-Toe). Cờ vua có không gian trạng thái 10^{40} và nhánh cây trò chơi 10^{123} , làm cho việc tìm kiếm trở nên cực kỳ tốn kém!

Công thức hóa Trò chơi 2 người

◇ Hai người chơi được đặt tên là **MAX** và **MIN**. MAX đi trước, và hai bên luân phiên nhau.

◇ Trò chơi được định nghĩa qua các thành phần:

- S_0 : Trạng thái ban đầu.
- $\text{Player}(s)$: Hàm trả về người đến lượt đi ở trạng thái s .
- $\text{Actions}(s)$: Danh sách các nước đi hợp lệ.
- $\text{Result}(s, a)$: Trạng thái mới (Transition model).
- $\text{TerminalTest}(s)$: Hàm kiểm tra trò chơi đã kết thúc chưa.
- $\text{Utility}(s, p)$: Hàm **Độ thỏa dụng (Hàm phần thưởng)**. Ví dụ: Cờ vua gán +1 cho thắng, -1 cho thua, và 0 cho hòa đối với người chơi p .

Cây Trò chơi (Game Tree)

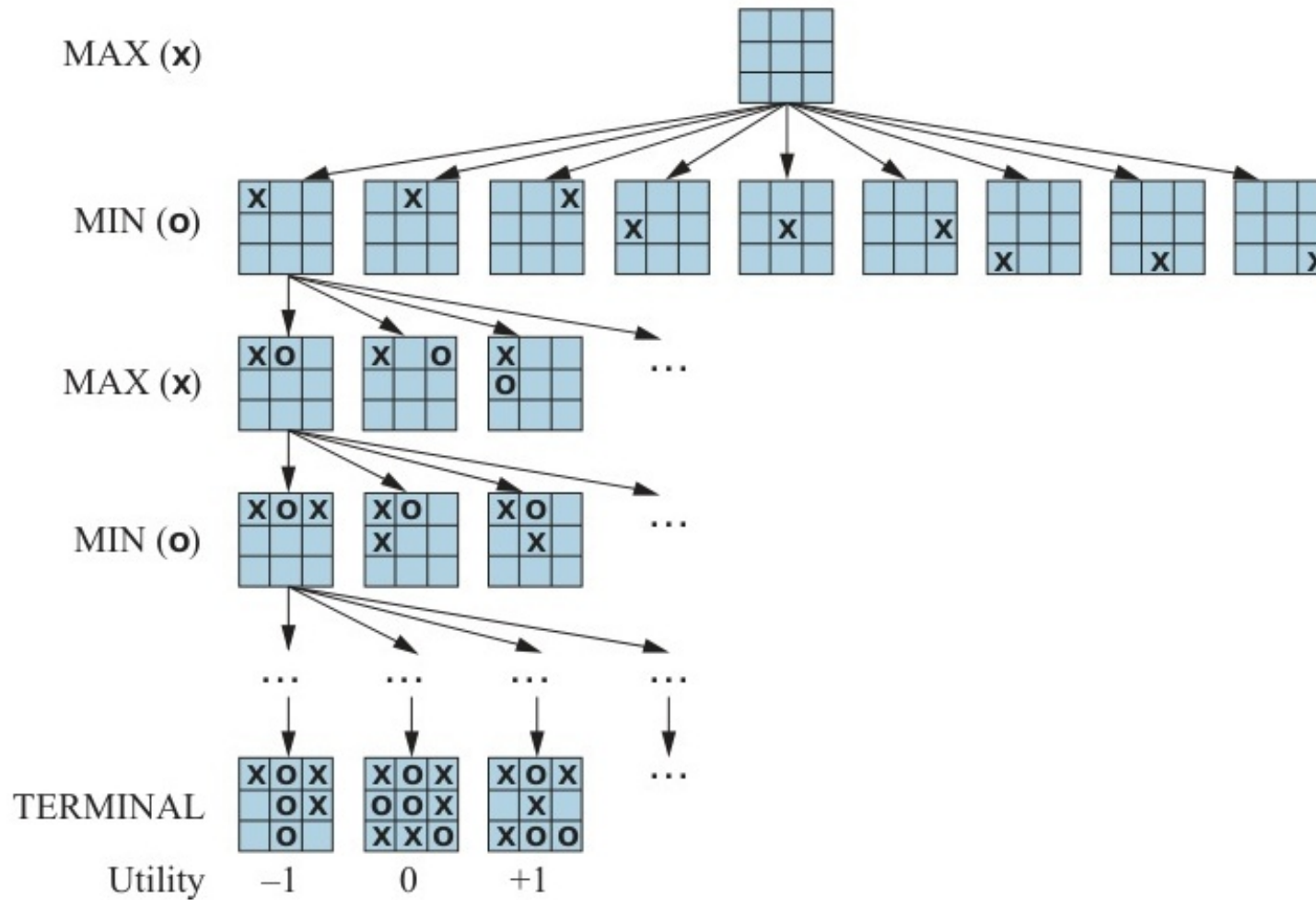


Figure 6.1: Mô phỏng 2 nước đi đầu tiên trong cây trò chơi Cờ caro (Tic-Tac-Toe).

6.2 Quyết định tối ưu: Thuật toán Minimax

◇ Giả sử cả hai người chơi đều chơi tối ưu (Không ai mắc sai lầm), MAX muốn tối đa hóa hàm Utility, còn MIN muốn tối thiểu hóa nó.

◇ Hàm **Giá trị Minimax (Minimax Value)** của một nút n : Là phần thưởng tốt nhất mà MAX có thể ép buộc đạt được từ nút n . Được tính đệ quy như sau:

- $\text{Minimax}(n) = \text{Utility}(n)$ (Nếu n là nút kết thúc).
- $\text{Minimax}(n) = \max_{s \in \text{Children}(n)} \text{Minimax}(s)$ (Nếu n là lượt của MAX).
- $\text{Minimax}(n) = \min_{s \in \text{Children}(n)} \text{Minimax}(s)$ (Nếu n là lượt của MIN).

Thuật toán Minimax

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

player $\leftarrow$ *game*.TO-MOVE(*state*)

value, move $\leftarrow$ MAX-VALUE(*game, state*)

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state, player*), *null*

v, move $\leftarrow -\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MIN-VALUE(*game, game*.RESULT(*state, a*))

if *v2* > *v* **then**

v, move $\leftarrow$ *v2, a*

return *v, move*

function MIN-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state, player*), *null*

v, move $\leftarrow +\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MAX-VALUE(*game, game*.RESULT(*state, a*))

if *v2* < *v* **then**

v, move $\leftarrow$ *v2, a*

return *v, move*

Figure 6.2: Thuật toán để tính toán nước đi tối ưu bằng cách sử dụng minimax.

Mô phỏng Thuật toán Minimax

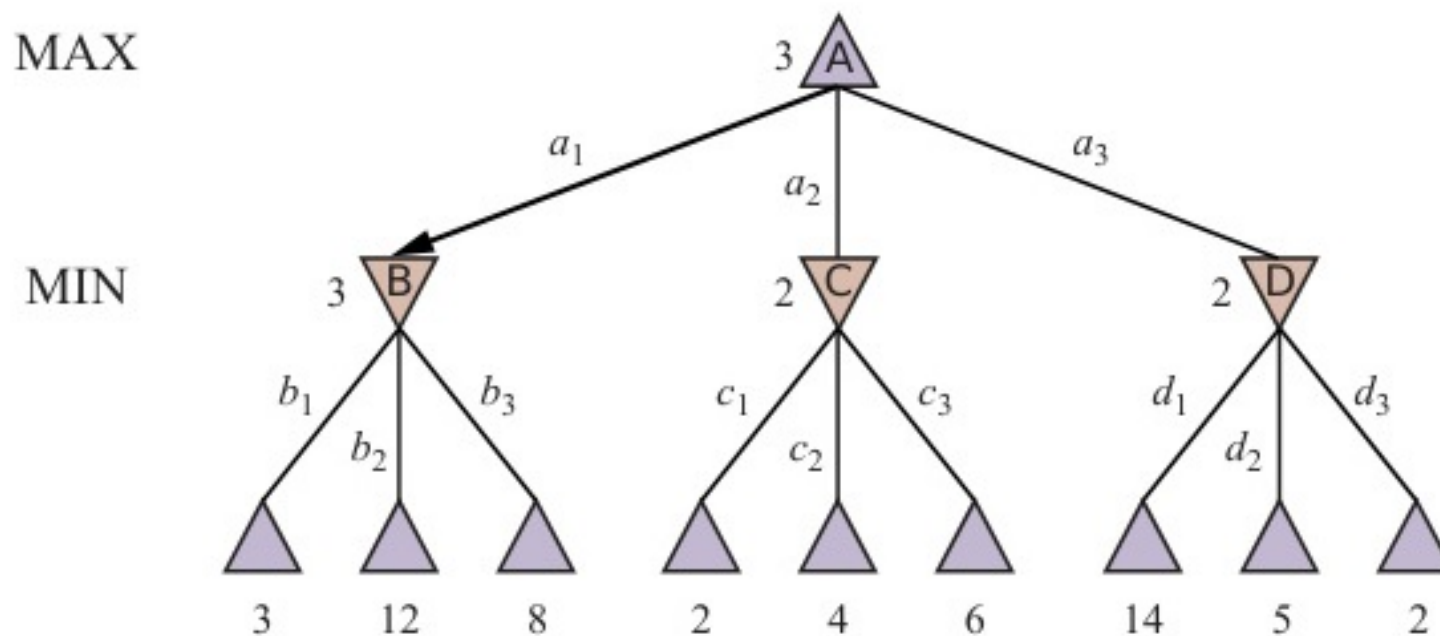


Figure 6.3: Hoạt động của Minimax: Gốc (MAX) sẽ chọn nước đi dẫn tới giá trị 3 (Cao nhất trong các lựa chọn ép buộc của MIN).

Đánh giá Thuật toán Minimax

◇ **Tính hoàn thiện (Completeness):** Có (Nếu cây trò chơi hữu hạn).

◇ **Tính tối ưu (Optimality):** Có (Chống lại một đối thủ chơi tối ưu).

◇ **Độ phức tạp Thời gian (Time):** $\mathcal{O}(b^m)$

- Với b là hệ số rẽ nhánh (số nước đi hợp lệ trung bình mỗi lượt), m là độ sâu tối đa của trò chơi.

◇ **Độ phức tạp Không gian (Space):** $\mathcal{O}(bm)$ (Khá tiết kiệm vì nó hoạt động giống DFS).

◇ **Vấn đề chí tử:** Đối với cờ vua, $b \approx 35$, $m \approx 100 \Rightarrow 35^{100}$ là con số khổng lồ vô tận, siêu máy tính cũng không đủ thời gian để tính xong một nước đi!

Mở rộng: Cây trò chơi nhiều người

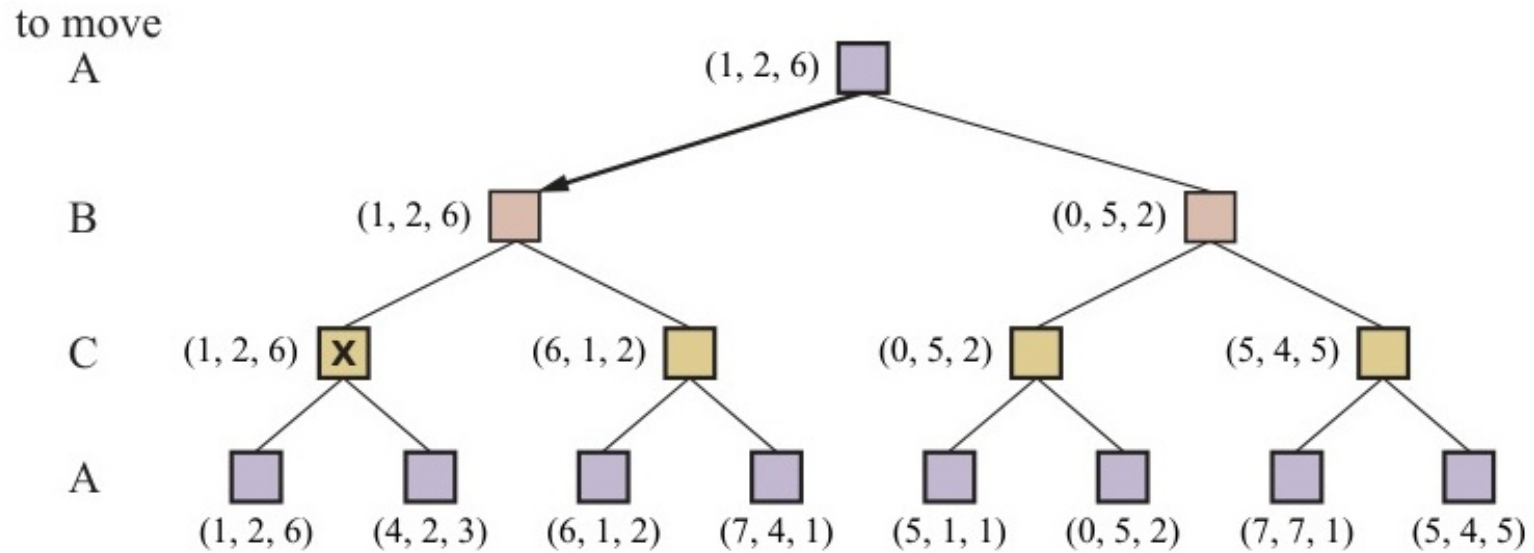


Figure 6.4: Ba ply đầu tiên của một cây trò chơi với ba người chơi (A, B, C).

6.3 Cắt tỉa Alpha-Beta

◇ Thuật toán Minimax duyệt toàn bộ cây, kể cả những nhánh chắc chắn không bao giờ được đối thủ lựa chọn. Ta có thể **Cắt bỏ (Prune)** các nhánh này để tiết kiệm thời gian mà **không làm thay đổi kết quả cuối cùng**.

◇ **Khái niệm cốt lõi của Cắt tỉa Alpha-Beta:**

- α (Alpha): Giá trị tốt nhất hiện tại dành cho MAX trên đường đi từ Gốc xuống. (Giá trị tối thiểu MAX chắc chắn nắm giữ).
- β (Beta): Giá trị tốt nhất hiện tại dành cho MIN trên đường đi từ Gốc xuống. (Giá trị tối đa MIN sẽ chịu đựng).

◇ **Điều kiện cắt tỉa:** Bất kỳ lúc nào $\alpha \geq \beta$, nhánh đang xét sẽ lập tức bị cắt bỏ (Không cần duyệt sâu thêm).

Minh họa Cắt tỉa Alpha-Beta

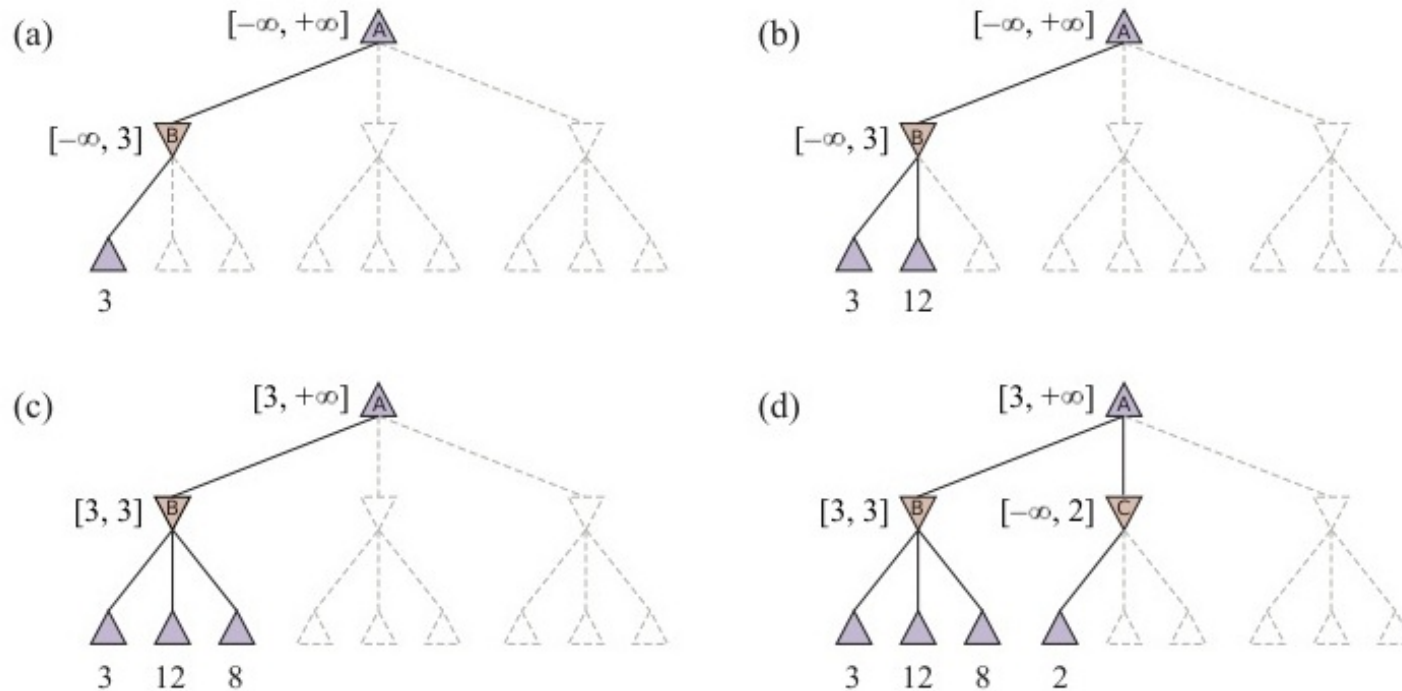


Figure 6.5: Mô phỏng cắt tỉa: Nhánh thứ 3 bị bỏ qua vì MIN chắc chắn sẽ ép giá trị xuống ≤ 2 , trong khi MAX đã có nhánh khác đảm bảo giá trị lớn hơn là 3.

Trường hợp Tổng quát cho Cắt tỉa Alpha-Beta

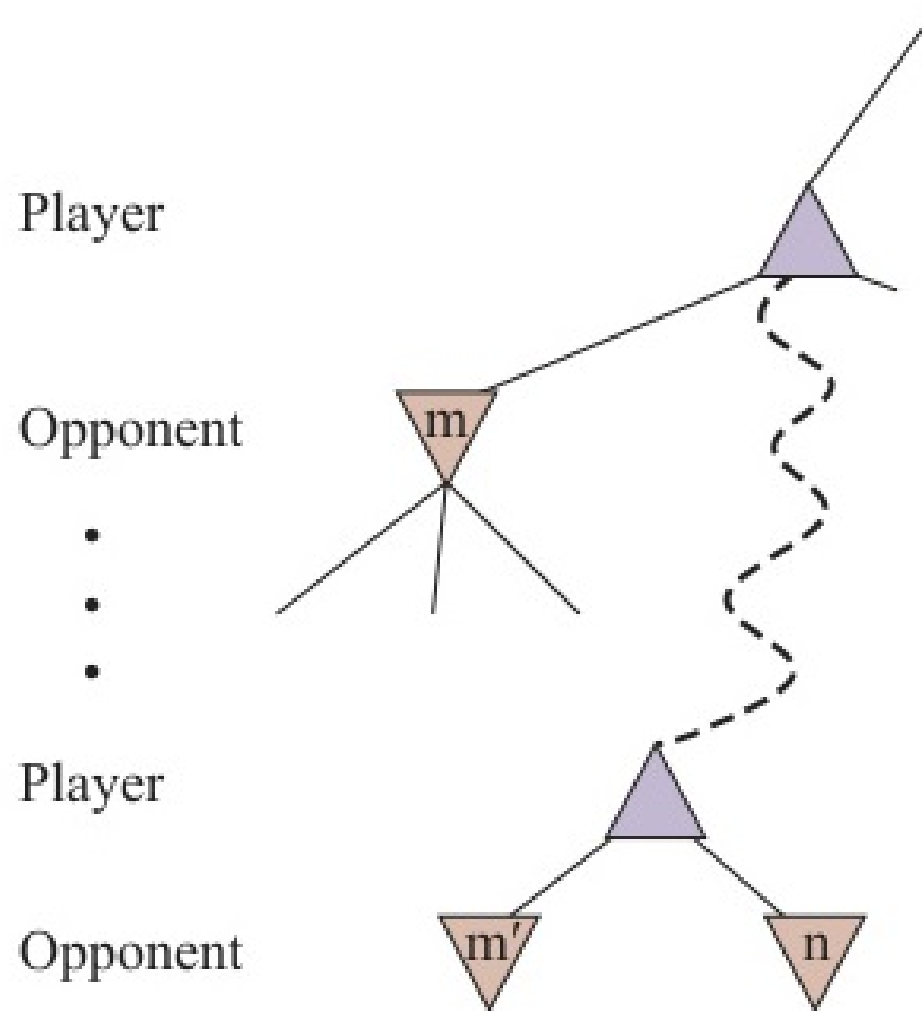


Figure 6.6: Trường hợp tổng quát cho việc cắt tỉa alpha-beta trên cây trò chơi.

Thuật toán Tìm kiếm Alpha-Beta

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action

player $\leftarrow$ *game*.TO-MOVE(*state*)

value, *move* $\leftarrow$ MAX-VALUE(*game*, *state*, $-\infty$, $+\infty$)

return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state*, *player*), null

v $\leftarrow -\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MIN-VALUE(*game*, *game*.RESULT(*state*, *a*), α , β)

if *v2* > *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

$\alpha \leftarrow$ MAX(α , *v*)

if *v* $\geq$ β **then return** *v*, *move*

return *v*, *move*

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state*, *player*), null

v $\leftarrow +\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MAX-VALUE(*game*, *game*.RESULT(*state*, *a*), α , β)

if *v2* < *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

$\beta \leftarrow$ MIN(β , *v*)

if *v* $\leq$ α **then return** *v*, *move*

return *v*, *move*

Figure 6.7: Thuật toán tìm kiếm alpha-beta.

Hiệu quả của Cắt tỉa Alpha-Beta

◇ Cắt tỉa Alpha-Beta **không** ảnh hưởng đến độ chính xác của nghiệm cuối cùng (Nó luôn trả về cùng một nước đi hết như Minimax).

◇ **Tầm quan trọng của Thứ tự duyệt (Move Ordering):**

- Nếu đánh giá các nhánh một cách ngẫu nhiên, độ phức tạp sẽ giảm xuống khoảng $\mathcal{O}(b^{3m/4})$.
- Nếu sắp xếp các nước đi tốt nhất lên trước (Perfect ordering), độ phức tạp thời gian giảm đột phá xuống chỉ còn $\mathcal{O}(b^{m/2})$.

◇ **Hệ quả to lớn:** Điều này có nghĩa Alpha-Beta có thể giải quyết các trò chơi ở độ sâu **gấp đôi** so với Minimax truyền thống trong cùng một giới hạn thời gian!

6.4 Quyết định Thời gian thực Không hoàn hảo

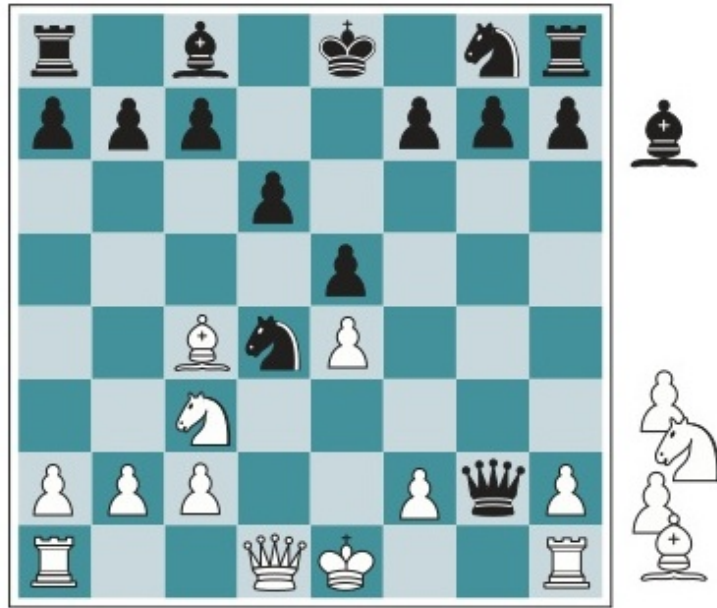
◇ Dù có cắt tỉa Alpha-Beta, ta vẫn không thể chạm tới độ sâu tối đa (Lá kết thúc) của cờ vua. Ta bắt buộc phải **cắt ngang quá trình tìm kiếm sớm (Cutoff test)**.

◇ Thay vì dùng hàm Utility (chỉ xác định ở nút lá), ta sử dụng hàm **Đánh giá Heuristic (Heuristic Evaluation Function)** $Eval(s)$ để chấm điểm lợi thế của trạng thái hiện tại đang xét.

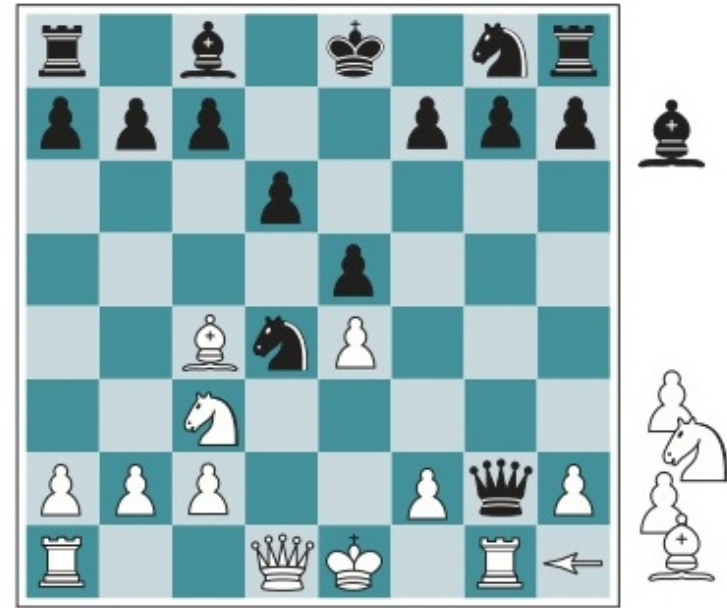
◇ **Yêu cầu của hàm Đánh giá:**

- Phải đồng nhất (order) với hàm Utility ở các nút kết thúc thực sự.
- Tính toán cực kỳ nhanh.
- Phản ánh chuẩn xác xác suất chiến thắng. VD trong cờ vua: Điểm thế cờ = Tổng điểm quân ta – Tổng điểm quân địch (Hậu=9, Xe=5, Mã=3, Tốt=1).

Đánh giá Heuristic trong Cờ Vua



(a) White to move



(b) White to move

Figure 6.8: Hai vị trí cờ vua chỉ khác nhau ở vị trí của quân xe. Đánh giá Heuristic cần nhạy bén với sự khác biệt nhỏ nhưng quan trọng này.

Cạm bẫy của Heuristic Minimax

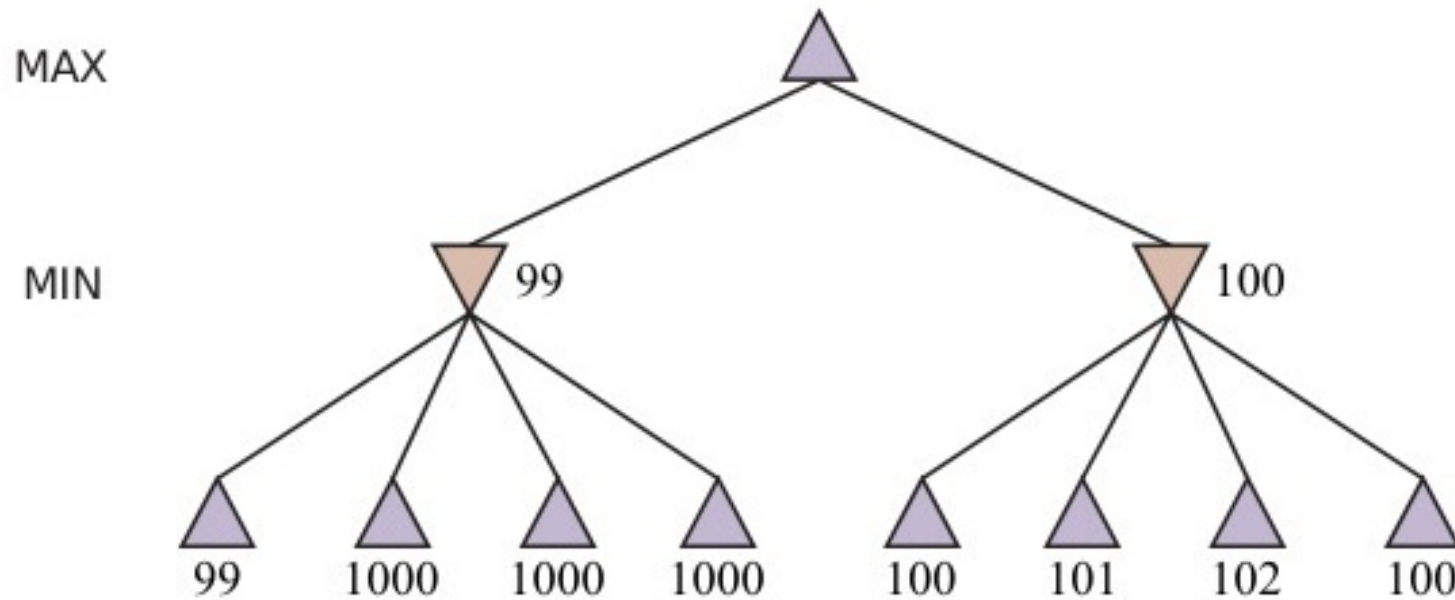


Figure 6.9: Một cây trò chơi hai ply mà heuristic minimax có thể mắc lỗi trong việc ra quyết định do đánh giá không chính xác ở độ sâu cạn.

Hiệu ứng Đường chân trời (Horizon Effect)

◇ **Horizon Effect:** Khi máy tính bị giới hạn độ sâu (VD: 8 nước đi), nó có thể cố tình đẩy các nước đi thiệt hại nặng nề lùi về sau nước thứ 9, tự tạo ảo giác rằng bản thân đang an toàn.

◇ **Giải pháp - Quiescence Search (Tìm kiếm tĩnh lặng):**

- Máy tính chỉ được phép dừng đánh giá (Cutoff) ở những trạng thái "tĩnh lặng" (không có những nước đi dữ dội có khả năng lật ngược thế cờ ngay sau đó).
- Nếu trạng thái đang có biến động đe dọa mạnh (VD: Bị chiếu tướng, Đổi quân liên hoàn), thuật toán tự động đào sâu thêm một vài nước (Quiescence search) cho đến khi thế cờ lắng xuống hoàn toàn.

Minh họa Hiệu ứng Đường chân trời

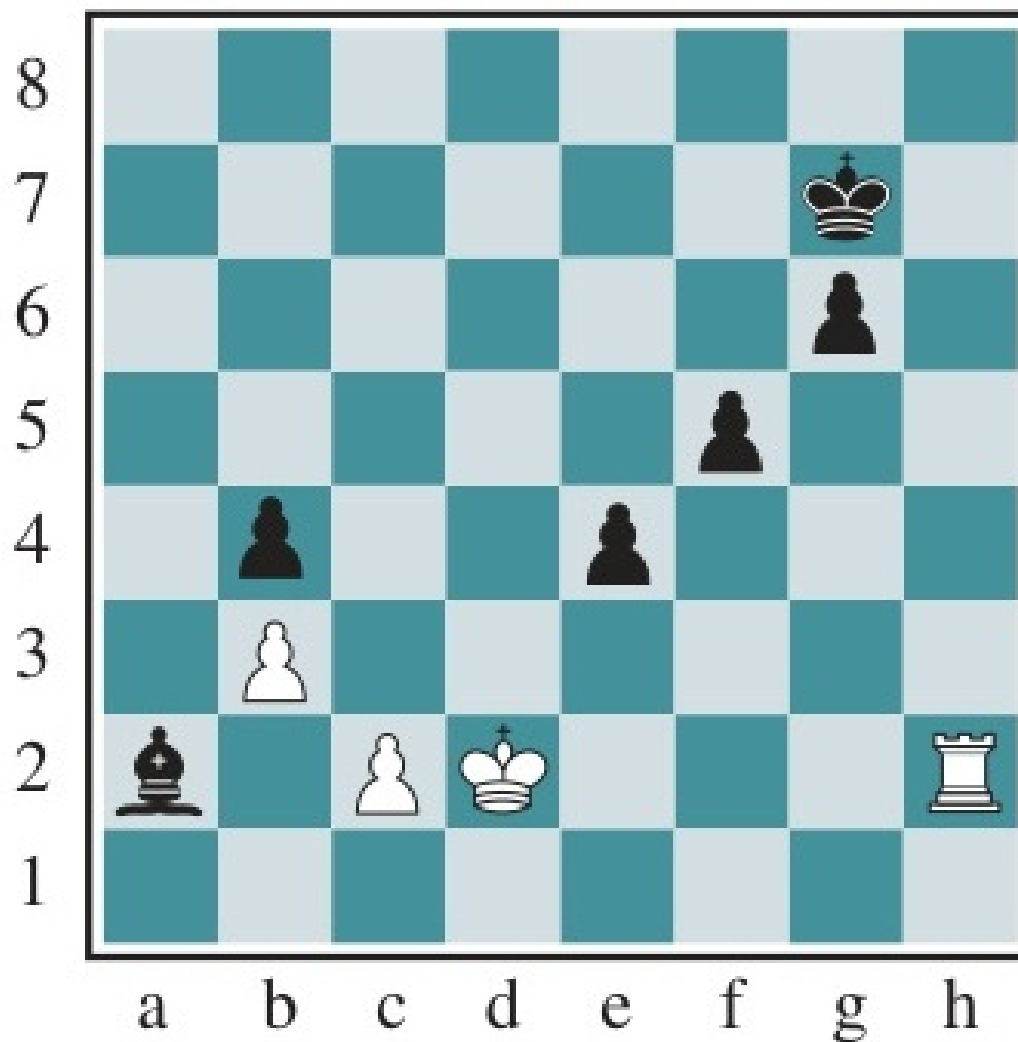


Figure 6.10: Hiệu ứng chân trời trong cờ vua: Máy tính đẩy nước đi thiệt hại về sau giới hạn tính toán.

Tìm kiếm Cây Monte Carlo (MCTS)

◇ Một phương pháp thay thế cực kỳ hiệu quả khi không thể viết hàm heuristic chính xác (VD: Cờ Vây) là **Tìm kiếm Cây Monte Carlo (MCTS)**.

◇ **Nguyên lý:** Chơi thử hàng vạn ván cờ ngẫu nhiên (Playout) từ trạng thái hiện tại để đánh giá xác suất chiến thắng.

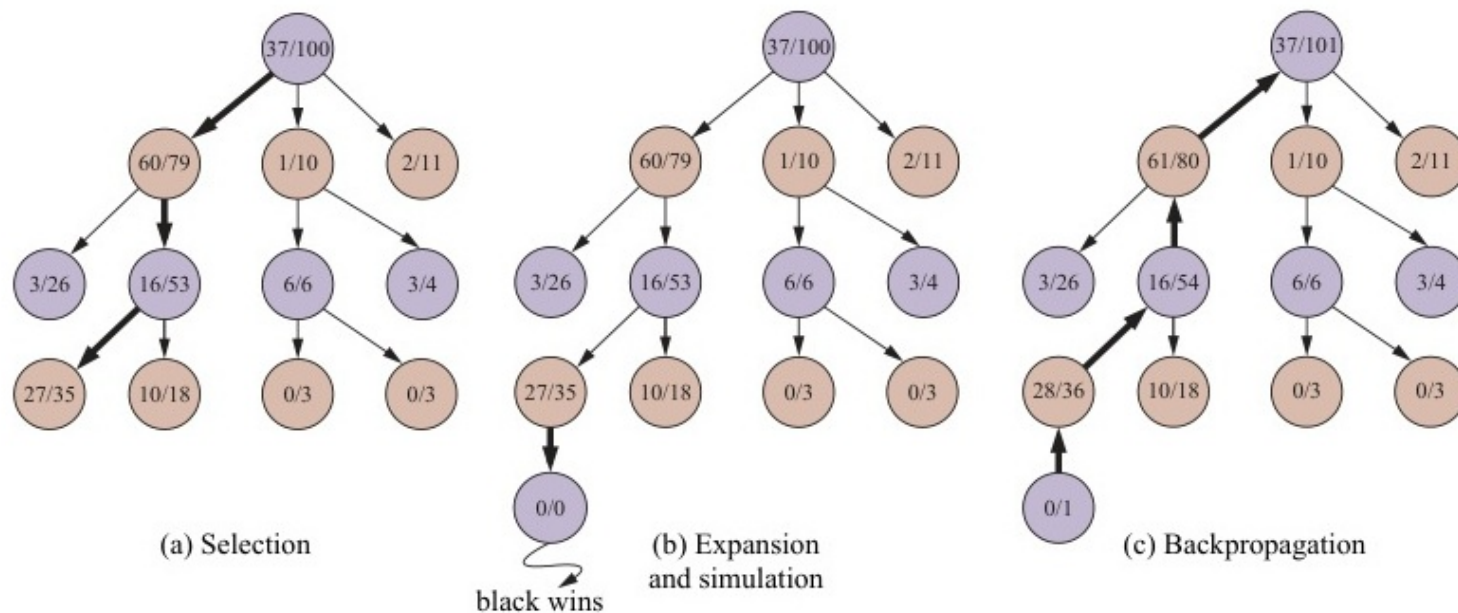


Figure 6.11: Quá trình chọn một nước đi bằng MCTS sử dụng số liệu lựa chọn UCT.

Thuật toán Tìm kiếm Cây Monte Carlo

```
function MONTE-CARLO-TREE-SEARCH(state) returns an action  
  tree  $\leftarrow$  NODE(state)  
  while IS-TIME-REMAINING() do  
    leaf  $\leftarrow$  SELECT(tree)  
    child  $\leftarrow$  EXPAND(leaf)  
    result  $\leftarrow$  SIMULATE(child)  
    BACK-PROPAGATE(result, child)  
  return the move in ACTIONS(state) whose node has highest number of playouts
```

Figure 6.12: Thuật toán tìm kiếm cây Monte Carlo hoàn chỉnh.

6.5 Trò chơi Ngẫu nhiên (Stochastic Games)

- ◇ Trong thực tế có nhiều yếu tố may rủi tác động (VD: Đồ xúc xắc trong cờ cá ngựa, Backgammon).
- ◇ Biểu diễn qua **Nút Xác suất (Chance nodes)**: Được chèn vào giữa lượt đi của MAX và MIN trên cây trò chơi.
- ◇ **Hàm Expectiminimax**: Thay vì lấy giá trị Max hay Min, ta phải lấy **Giá trị Kỳ vọng (Expected Value)** tính trên mọi mặt của xúc xắc:

$$\text{Expected}(n) = \sum_i P(d_i) \times \text{Minimax}(\text{Result}(n, d_i))$$

Ví dụ: Cờ tàn cáo (Backgammon)

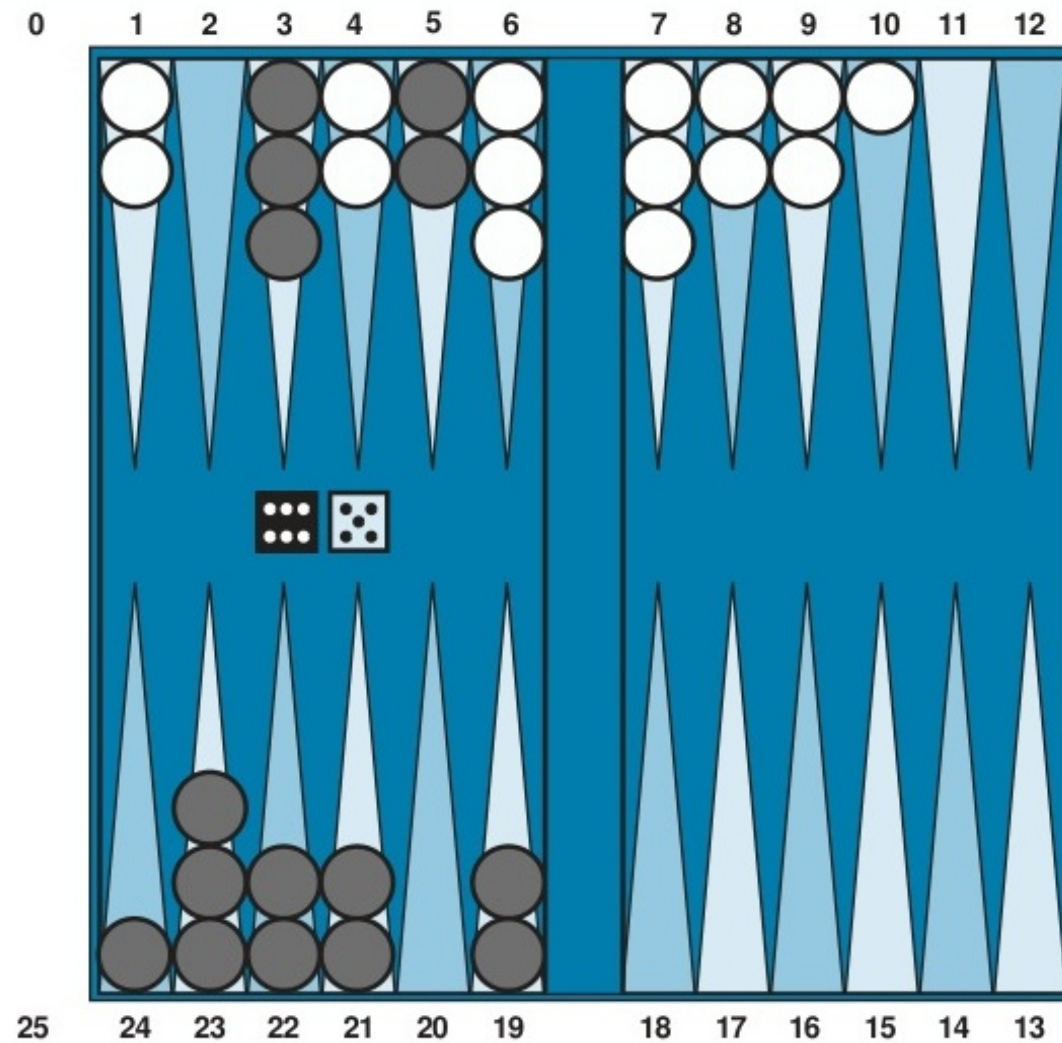


Figure 6.13: Một vị trí diễn hình trong trò chơi có yếu tố ngẫu nhiên (đổ xúc xắc).

Cây trò chơi Expectiminimax

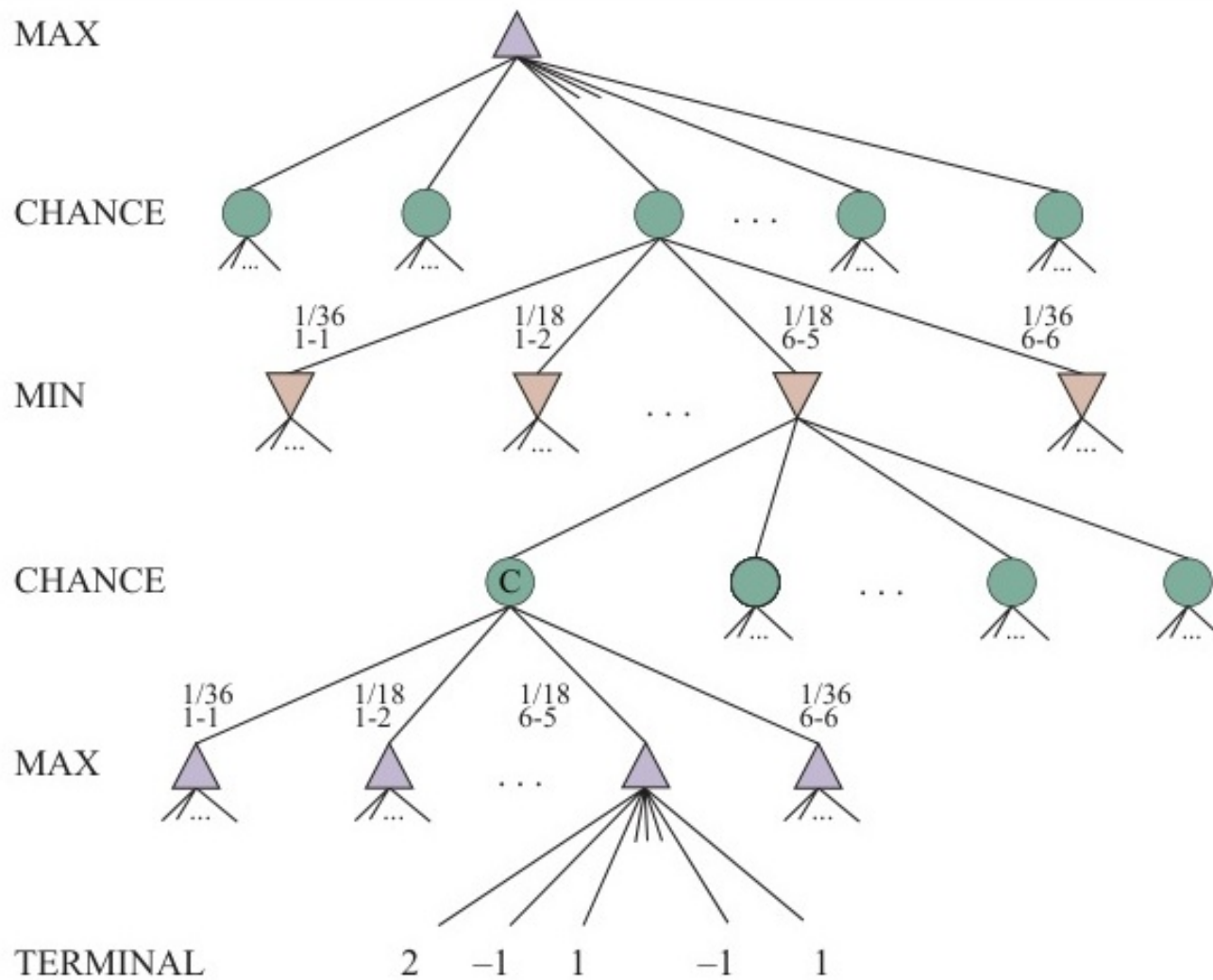


Figure 6.14: Mô phỏng cây trò chơi Expectiminimax với các nút tròn xúc xắc xen ngang.

Tính nhạy cảm của Expectiminimax

◇ Khác với Minimax thông thường (chỉ quan tâm thứ tự lớn nhỏ), Expectiminimax **rất nhạy cảm với giá trị tuyệt đối** của hàm đánh giá (do phải tính trung bình). Một phép biến đổi bảo toàn thứ tự có thể làm thay đổi hoàn toàn quyết định.

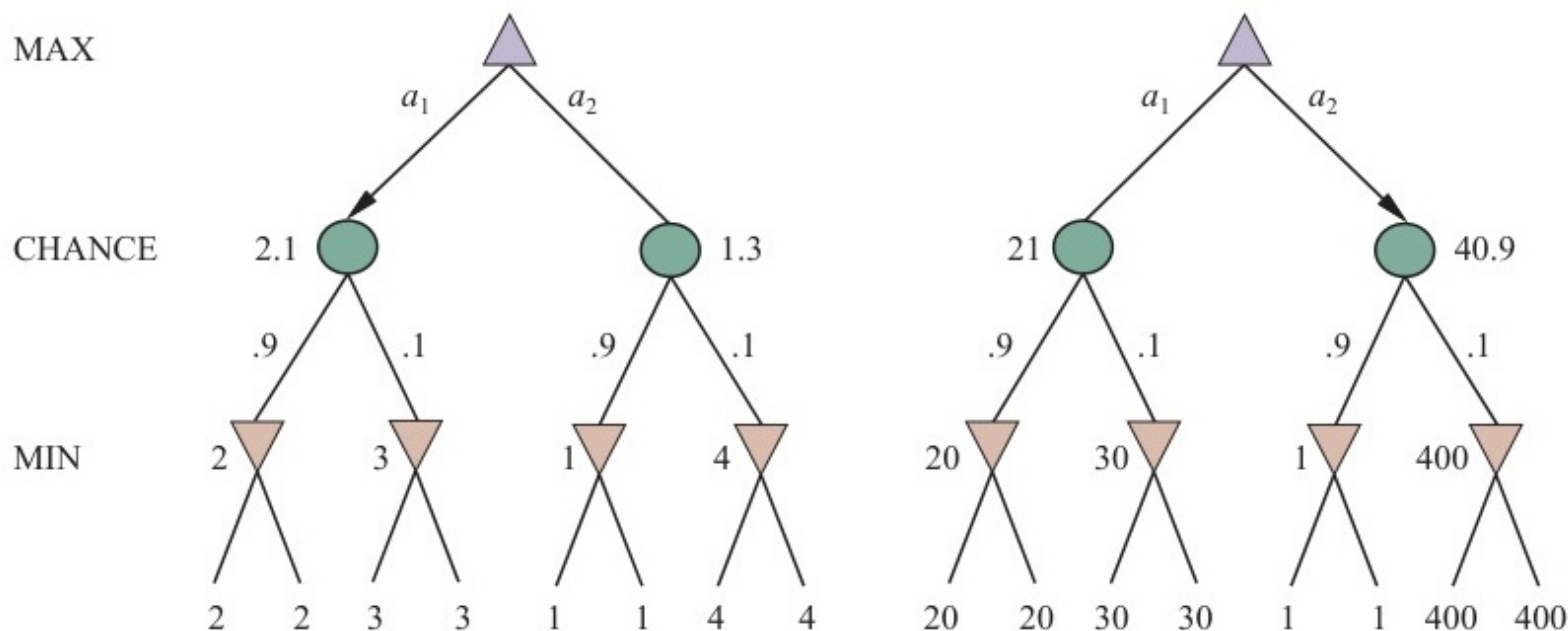


Figure 6.15: Một phép biến đổi bảo toàn thứ tự trên các giá trị lá có thể làm thay đổi nước đi tốt nhất.

6.6 Khuyết thiếu thông tin (Partially Observable)

- ◇ Diễn hình như các trò chơi bài (Poker), người chơi không thể nhìn thấy bài của đối phương (Khuyết thiếu thông tin).
- ◇ Trò chơi lúc này không còn là một cây trạng thái hiển nhiên, mà phân mảnh thành các **Tập hợp thông tin (Information sets)**.
- ◇ **Chiến lược thuần túy (Pure strategy)** không còn hiệu quả: Tác nhân bắt buộc phải thi triển **Chiến lược hỗn hợp (Mixed strategy)** (lựa chọn hành động ngẫu nhiên với một xác suất được tính toán tỉ mỉ) để không bị đối thủ bắt bài hay đọc vị (Kỹ năng Bluffing trong Poker).

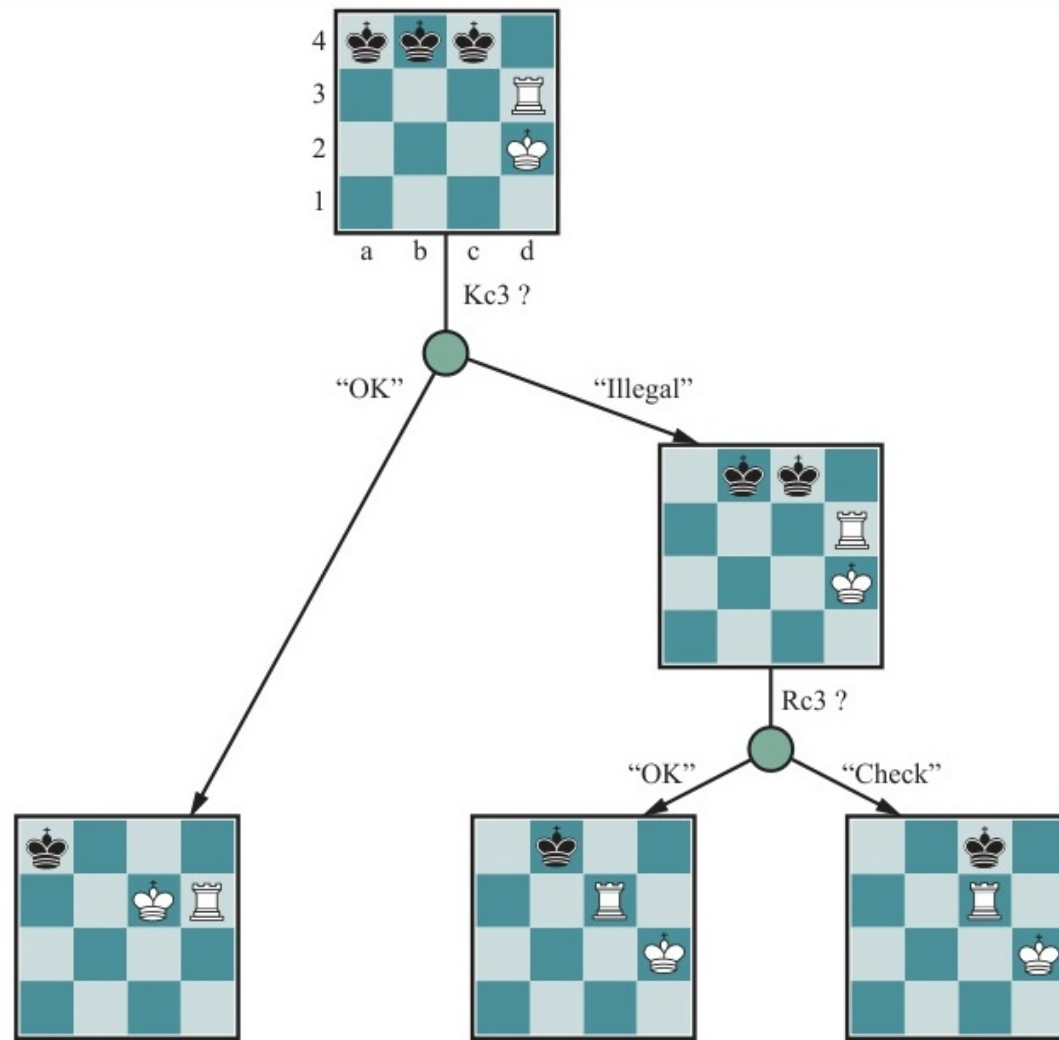


Figure 6.16: Tìm kiếm chiến lược trong điều kiện khuyết thiếu thông tin (VD: Trận cờ vua mù - Kriegspiel).

Tóm tắt Chương 6

- ◇ Trò chơi đối kháng là môi trường mà 2 tác nhân cạnh tranh trực tiếp qua hàm mục tiêu tổng bằng không.
- ◇ **Thuật toán Minimax:** Lựa chọn nước đi tối ưu nhất bằng cách giả định đối thủ cũng không bao giờ mắc sai lầm. Tuy nhiên, nó bị phê truất trong thực tế bởi độ phức tạp không gian và thời gian khổng lồ.
- ◇ **Cắt tỉa Alpha-Beta:** Là bản nâng cấp xuất chúng, loại bỏ vô số nhánh tính toán thừa thãi, giúp tăng độ sâu tính toán lên gấp đôi trong cùng một khoảng thời gian.
- ◇ **Hệ thống Thực tiễn:** Các AI siêu việt chơi cờ hiện đại (Deep Blue, AlphaGo) hoạt động nhờ sự kết hợp: Alpha-Beta sâu + Hàm Đánh giá Heuristic + Tìm kiếm Quiescence + Học tăng cường.